

"""

verrouillage_1_1_sweep_beta_tau.py
=====

Objectif : verrouiller le point 1.1 de l'audit V107+1.

Question : ($\beta=0.30$, $\tau=0.50$) sont-ils uniques, ou existe-t-il une famille continue (β , τ) donnant les mêmes propriétés observables ?

Stratégie Rasoir d'Ockham :

Chemin 1 – Sweep d_s sur grille (β , τ) : identifier le manifold $d_s = 3$.

Chemin 2 – Sweep gap n_{tri} Blob/null : identifier le manifold à gap max.

Chemin 3 – Sweep dispersion $\bar{D}$: identifier le manifold à $\bar{D}$ canonique.

Si les 3 manifolds se croisent en un point unique ($\beta=0.30$, $\tau=0.50$) :

→ unicité validée [T].

Si les 3 manifolds définissent une courbe continue :

→ famille équivalente, à documenter [I].

Si les 3 manifolds ne se croisent pas du tout :

→ ($\beta=0.30$, $\tau=0.50$) ad hoc [FAIL partiel, à reformuler].

Note : sweep restreint à $N=200$, 3 seeds par cellule (coût raisonnable).

"""

```
import numpy as np
import math
import time
import json
from collections import deque
from scipy.spatial import KDTree
from scipy.linalg import eigvalsh
from scipy.stats import linregress

# Paramètres canoniques (référence)
KAPPA_0 = 8
BETA_CANON = 0.30
TAU_CANON = 0.50

# Grille du sweep
BETA_RANGE = np.array([0.10, 0.20, 0.30, 0.40, 0.50])
TAU_RANGE = np.array([0.30, 0.40, 0.50, 0.60, 0.70])

# Paramètres MCMC réduits pour coût raisonnable
N_TEST = 200
N_SEEDS = 3
```

```

BURN_FACTOR = 50    # Réduit de 100
PROD_FACTOR = 100   # Réduit de 200

# Facteurs correctifs (issus de calibration_ds.py validée)
BIAS_EIGVALS = 1.189

# =====
# CONSTRUCTION INITIALE (port O2v4 validé)
# =====

def _rips_radius(N, kappa=KAPPA_0):
    return (kappa / ((N-1) * (4/3) * math.pi)) ** (1/3)

def _E0(N, kappa=KAPPA_0):
    return int(N * kappa / 2)

def _lcc_size(adj, N):
    visited = [False]*N
    best = 0
    for s in range(N):
        if not visited[s]:
            sz = 0; q = deque([s]); visited[s] = True
            while q:
                v = q.popleft(); sz += 1
                for w in adj[v]:
                    if not visited[w]: visited[w] = True; q.append(w)
            best = max(best, sz)
    return best

def _build_initial(N, seed):
    r = _rips_radius(N)
    r_p = 1.5 * r
    E0 = _E0(N)
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0.0, 1.0, (N, 3))
    tree = KDTree(pts)
    adj = [set() for _ in range(N)]
    for i, j in tree.query_pairs(r):
        adj[i].add(j); adj[j].add(i)
    pool = list(tree.query_pairs(r_p))
    rng2 = np.random.default_rng(seed + 50_000)
    K = sum(len(adj[i]) for i in range(N)) // 2
    att = 0; max_att = 10 * abs(E0 - K) + 5000
    while K != E0 and att < max_att:
        att += 1
        if K < E0:

```

```

        e = pool[rng2.integers(len(pool))]; u, v = e
        if v not in adj[u]: adj[u].add(v); adj[v].add(u); K += 1
    else:
        edges = [(i,j) for i in range(N) for j in adj[i] if j>i]
        if not edges: break
        e = edges[rng2.integers(len(edges))]; u, v = e
        adj[u].discard(v); adj[v].discard(u)
        if _lcc_size(adj, N) == N: K -= 1
        else: adj[u].add(v); adj[v].add(u)
    return pts, adj, pool, E0

def _count_triangles(adj, N):
    n = 0
    for i in range(N):
        for j in adj[i]:
            if j > i:
                n += len([k for k in adj[i] & adj[j] if k > j])
    return n

def compute_S(adj, n_tri, N, beta):
    degs = np.array([len(adj[i]) for i in range(N)], float)
    D = degs.std() / (degs.mean() + 1e-8)
    return math.log(1 + n_tri) - beta * D

def _fast_still_connected(adj, u, v, N):
    visited = {u}; q = deque([u])
    while q:
        node = q.popleft()
        for w in adj[node]:
            if w in visited: continue
            if node == u and w == v: continue
            if node == v and w == u: continue
            visited.add(w); q.append(w)
            if w == v: return True
    return False

def run_mcmc(pts, adj_init, pool, N, E0, seed, beta, tau):
    """MCMC à ( $\beta$ ,  $\tau$ ) variables."""
    tau_eff = tau / N
    rng = np.random.default_rng(seed + 200_000)
    adj = [set(a) for a in adj_init]
    edge_list = [(i,j) for i in range(N) for j in adj[i] if j>i]
    n_tri = _count_triangles(adj, N)

```

```

S_curr = compute_S(adj, n_tri, N, beta)
n_steps = (BURN_FACTOR + PROD_FACTOR) * N
u_arr = rng.random(size=n_steps)
pool_arr = np.array(pool, dtype=np.int32)
n_pool = len(pool_arr)
n_accept = 0
for step in range(n_steps):
    if not edge_list: break
    idx_out = int(rng.integers(len(edge_list)))
    u, v = edge_list[idx_out]
    e_in = None
    for _ in range(30):
        idx_in = int(rng.integers(n_pool))
        a, b = int(pool_arr[idx_in,0]), int(pool_arr[idx_in,1])
        if b not in adj[a]: e_in = (a,b); break
    if e_in is None: continue
    a, b = e_in
    if len(adj[u]) <= 2 or len(adj[v]) <= 2:
        if not _fast_still_connected(adj, u, v, N): continue
    tri_lost = len(adj[u] & adj[v])
    adj[u].discard(v); adj[v].discard(u)
    adj[a].add(b); adj[b].add(a)
    tri_gained = len(adj[a] & adj[b])
    n_tri_new = n_tri - tri_lost + tri_gained
    S_new = compute_S(adj, n_tri_new, N, beta)
    dS = S_new - S_curr
    if dS >= 0 or u_arr[step] < math.exp(dS / tau_eff):
        n_tri = n_tri_new; S_curr = S_new
        n_accept += 1
        edge_list[idx_out] = (a, b)
    else:
        adj[u].add(v); adj[v].add(u); adj[a].discard(b); adj[b].discard(a)
return adj, n_tri, n_accept / n_steps

```

```

# =====
# OBSERVABLES
# =====

```

```

def measure_ds_eigvals(adj_sets, N):
    """d_s par comptage eigenvalues, corrigé du biais."""
    A = np.zeros((N, N), dtype=np.int8)
    for i in range(N):
        for j in adj_sets[i]:
            A[i, j] = 1
    deg = A.sum(axis=1).astype(np.float64)
    L = np.diag(deg) - A.astype(np.float64)

```

```

evals = np.sort(eigvalsh(L))
evals = evals[evals > 1e-9]
if len(evals) < 20: return None
lam_max = evals.max()
threshold = 0.15 * lam_max
n_low = int((evals <= threshold).sum())
if n_low < 10: n_low = max(20, len(evals)//4)
lam_arr = evals[:n_low]
N_arr = np.arange(1, n_low + 1)
res = linregress(np.log(lam_arr), np.log(N_arr))
return float(2 * res.slope * BIAS_EIGVALS) # corrigé

def measure_D(adj_sets, N):
    degs = np.array([len(adj_sets[i]) for i in range(N)], float)
    return degs.std() / (degs.mean() + 1e-8)

# =====
# SWEEP
# =====

def run_sweep():
    results = {}
    t_total_start = time.time()
    total_runs = len(BETA_RANGE) * len(TAU_RANGE) * N_SEEDS
    run_idx = 0

    print(f"=== Sweep ( $\beta$ ,  $\tau$ ) à N={N_TEST}, {N_SEEDS} seeds par cellule ===")
    print(f"Total runs : {total_runs}, paramètres MCMC : burn={BURN_FACTOR}N, pro
    print(f"Grille :  $\beta \in \{list(BETA\_RANGE)\}$ ,  $\tau \in \{list(TAU\_RANGE)\}$ \n")
    print(f"{' $\beta$ ':>6}  {' $\tau$ ':>6}  {'{d_s}':>10}  {' $\sigma_{ds}$ ':>8}  {'{n_tri}':>10}  {'{D
    print('-' * 70)

    for beta in BETA_RANGE:
        for tau in TAU_RANGE:
            ds_list = []
            ntri_list = []
            D_list = []
            acc_list = []

            for seed in range(N_SEEDS):
                run_idx += 1
                pts, adj_init, pool, E0 = _build_initial(N_TEST, seed)
                K_init = sum(len(adj_init[i]) for i in range(N_TEST)) // 2
                if _lcc_size(adj_init, N_TEST) / N_TEST < 0.90 or K_init != E0:
                    continue

```

```

        adj, n_tri, acc = run_mcmc(pts, adj_init, pool, N_TEST, E0, seed,
        ds = measure_ds_eigvals(adj, N_TEST)
        D = measure_D(adj, N_TEST)
        if ds is not None:
            ds_list.append(ds)
        ntri_list.append(n_tri)
        D_list.append(D)
        acc_list.append(acc)

    if ds_list:
        ds_mean = float(np.mean(ds_list))
        ds_std = float(np.std(ds_list))
        ntri_mean = float(np.mean(ntri_list))
        D_mean = float(np.mean(D_list))
        acc_mean = float(np.mean(acc_list))

        results[f"{beta}_{tau}"] = {
            'beta': float(beta), 'tau': float(tau),
            'd_s_mean': ds_mean, 'd_s_std': ds_std,
            'n_tri_mean': ntri_mean,
            'D_mean': D_mean,
            'acc_mean': acc_mean,
            'n_valid_seeds': len(ds_list)
        }

        canon_marker = " ← canonique" if (beta == BETA_CANON and tau == T
        print(f"{beta:>6.2f} {tau:>6.2f} {ds_mean:>10.3f} {ds_std:>8.3
              f"{ntri_mean:>10.0f} {D_mean:>8.4f} {acc_mean:>8.3f}{cano

    elapsed = time.time() - t_total_start
    print(f"\nTotal time: {elapsed:.1f}s")
    return results

def analyze(results):
    """Analyse Rasoir d'Ockham."""
    print(f"\n{'='*70}")
    print("ANALYSE RASOIR D'OCKHAM – Manifolds dans ( $\beta$ ,  $\tau$ )")
    print(f"\n{'='*70}")

    points = list(results.values())
    ds_arr = np.array([p['d_s_mean'] for p in points])
    ntri_arr = np.array([p['n_tri_mean'] for p in points])
    D_arr = np.array([p['D_mean'] for p in points])

    canon_key = f"{BETA_CANON}_{TAU_CANON}"
    canon = results.get(canon_key)

```

```

if canon is None:
    print("ERREUR : point canonique manquant.")
    return

print(f"\nValeurs au point canonique ( $\beta$ ={BETA_CANON},  $\tau$ ={TAU_CANON}) :")
print(f"  d_s    = {canon['d_s_mean']:.3f}  $\pm$  {canon['d_s_std']:.3f}")
print(f"  n_tri  = {canon['n_tri_mean']:.0f}")
print(f"  D      = {canon['D_mean']:.4f}")
print(f"  acc    = {canon['acc_mean']:.3f}")

# Chemin 1 : points où d_s  $\approx$  3 (à 1 $\sigma$  près du canonique)
ds_target = canon['d_s_mean']
ds_tol = max(2 * canon['d_s_std'], 0.20)
near_ds = [p for p in points if abs(p['d_s_mean'] - ds_target) < ds_tol]
print(f"\n[Chemin 1] Points où d_s  $\approx$  {ds_target:.2f} ( $\pm$ {ds_tol:.2f}) :")
for p in near_ds:
    marker = "  $\leftarrow$  CANON" if (p['beta']==BETA_CANON and p['tau']==TAU_CANON) else ""
    print(f"  ( $\beta$ ={{p['beta']:.2f}},  $\tau$ ={{p['tau']:.2f}}) : d_s = {{p['d_s_mean']:.3f}}{marker}")

# Chemin 2 : points où n_tri  $\approx$  canonique (à 10% près)
ntri_target = canon['n_tri_mean']
near_ntri = [p for p in points if abs(p['n_tri_mean'] - ntri_target) / ntri_target < 0.10]
print(f"\n[Chemin 2] Points où n_tri  $\approx$  {ntri_target:.0f} ( $\pm$ 10%) :")
for p in near_ntri:
    marker = "  $\leftarrow$  CANON" if (p['beta']==BETA_CANON and p['tau']==TAU_CANON) else ""
    print(f"  ( $\beta$ ={{p['beta']:.2f}},  $\tau$ ={{p['tau']:.2f}}) : n_tri = {{p['n_tri_mean']:.0f}}{marker}")

# Chemin 3 : points où D  $\approx$  canonique (à 10% près)
D_target = canon['D_mean']
near_D = [p for p in points if abs(p['D_mean'] - D_target) / D_target < 0.10]
print(f"\n[Chemin 3] Points où D  $\approx$  {D_target:.4f} ( $\pm$ 10%) :")
for p in near_D:
    marker = "  $\leftarrow$  CANON" if (p['beta']==BETA_CANON and p['tau']==TAU_CANON) else ""
    print(f"  ( $\beta$ ={{p['beta']:.2f}},  $\tau$ ={{p['tau']:.2f}}) : D = {{p['D_mean']:.4f}}{marker}")

# Intersection des 3 chemins
keys_ds = set(f"{{p['beta']}}_{{p['tau']}}" for p in near_ds)
keys_ntri = set(f"{{p['beta']}}_{{p['tau']}}" for p in near_ntri)
keys_D = set(f"{{p['beta']}}_{{p['tau']}}" for p in near_D)
intersection = keys_ds & keys_ntri & keys_D

print(f"\n{'='*70}")
print("VERDICT")
print(f"{'='*70}")
print(f"  Intersection des 3 chemins : {len(intersection)} points")
for k in intersection:
    p = results[k]

```

```

marker = " ← CANON" if k == canon_key else ""
print(f"      ( $\beta$ ={p['beta']:.2f},  $\tau$ ={p['tau']:.2f}){marker}")

if len(intersection) == 1 and canon_key in intersection:
    print("\n → UNICITÉ : seul ( $\beta$ =0.30,  $\tau$ =0.50) satisfait les 3 critères.")
    print(" → Point 1.1 [T] : paramètres canoniques uniquement validés.")
elif len(intersection) > 1:
    print(f"\n → FAMILLE ÉQUIVALENTE de {len(intersection)} points dans ( $\beta$ ,
    print(" → Point 1.1 [I] : famille admissible à documenter, choix canonique")
else:
    print("\n → AUCUNE INTERSECTION : choix canonique non triangulé par ces
    print(" → Point 1.1 [Q] : autres critères de fixation à identifier.")

if __name__ == "__main__":
    results = run_sweep()
    with open('/home/claude/sweep_beta_tau_results.json', 'w') as f:
        json.dump(results, f, indent=2)
    analyze(results)
    print(f"\nSauvegarde : /home/claude/sweep_beta_tau_results.json")

```